

Figure 3: Kubernetes cluster created at GKE

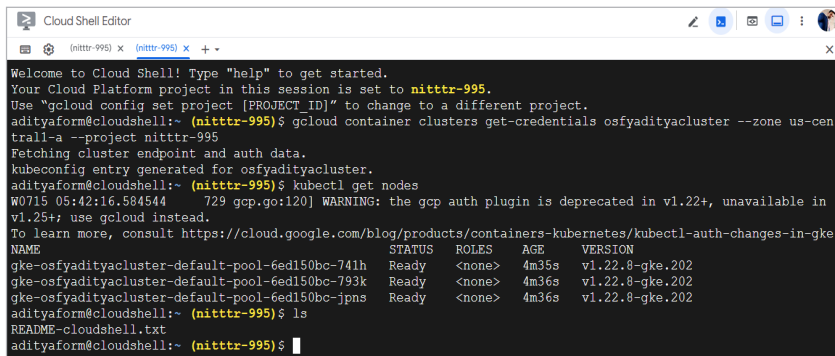


Figure 4: Kubernetes cluster running nodes

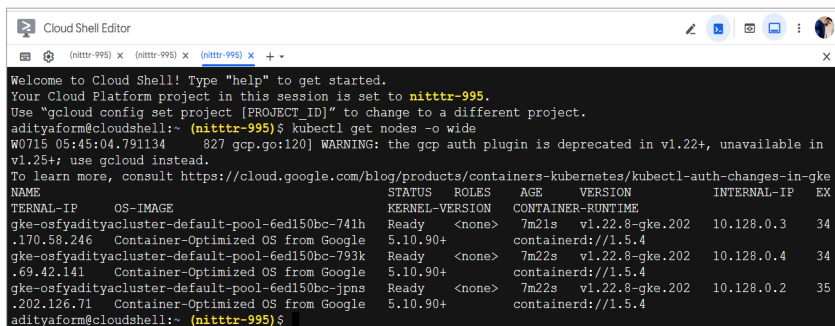


Figure 5: \$ kubectl get nodes --output wide command output

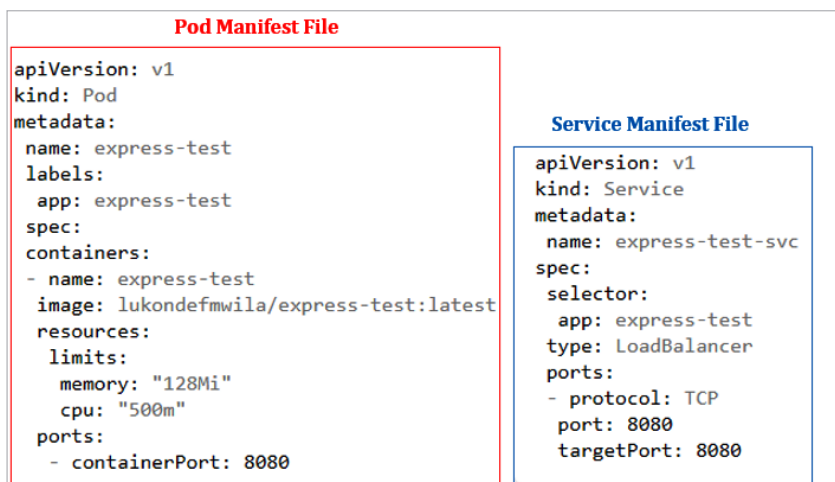


Figure 6: Configuration of Pod Manifest and Service Manifest files

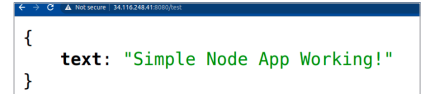


Figure 7: Node.js application running in the Kubernetes cluster

Manifest files have been deployed in this tutorial. Figure 6 displays the setup of these distributed files.

After creating these manifest files, you need to create two different resources in the cluster. The following commands are used for this purpose:

```
$ kubectl create -f service.yaml
```

```
$ kubectl create -f pod.yaml
```

You can verify that the pod is running as expected by running the `$ kubectl get pods` command in your terminal. After confirming that the pod is running, open a new tab in the browser and access the `/testroute` on your application. The running Node.js application is shown in Figure 7.

Managed Kubernetes clusters like GKE enable businesses to reap the benefits of operating their workloads in a secure and appropriately designed Kubernetes environment, without the need for a team of Kubernetes experts. Platforms like GKE that handle the grunt work of Kubernetes management while providing the advantages of container orchestration make the benefits of cloud native development accessible. **END** 🐧

References

- <https://kubernetes.io/docs/concepts/architecture/>
- <https://cloud.google.com/kubernetes-engine>

By: Dr Aditya Bhardwaj

The author is assistant professor in the School of Computer Science Engineering and Technology at Bennett University, Greater Noida. He is experienced in cloud computing and open source technology.